

# Função create\_sync\_packet():

```
def create_sync_packet(novo_vest_id, target_net_id, mac_address_int, data_ch, ctrl_ch):
    # Constrói o pacote de SYNC, junta os bits (Operadores << e |)
    net_id = 0b000
    byte0 = (net_id << 5) | (novo_vest_id & 0x1F)
    opcode = 0b0011 # Opcode 3 significa: "Configuração de Rede"
    byte1 = (opcode << 4) | (0b0 << 3) | (target_net_id & 0x07)
    mac_bytes = struct.pack('>I', mac_address_int) # Empacota o MAC Address em 4 bytes (Big-Endian)
    byte6 = ((data_ch & 0x07) << 5) | (0b0 << 4) | ((ctrl_ch & 0x07) << 1) | 0b0
    return bytes([byte0, byte1]) + mac_bytes + bytes([byte6])
```

```
byte0 = (net_id << 5) | (novo_vest_id & 0x1F)
```

- **novo\_vest\_id & 0x1F:** O **0x1F** em binário é **0001 1111** (5 bits a 1). Isto é uma máscara de segurança, para garantirmos que qualquer que seja o ID dado ao coleto, ele corta sempre o número para caber apenas em 5 bits.
- **net\_id << 5:** O NetID tem 3 bits (ex: 000). O comando **<< 5** empurra esses 3 bits para a esquerda, e adiciona 5 zeros à direita. Fica com algo tipo: **000x xxxx**.
- **Comando (|):** Junta tudo para ficarmos com algo do tipo **[3 bits de NetID] [5 bits de VestID]**.

```
opcode = 0b0011
byte1 = (opcode << 4) | (0b0 << 3) | (target_net_id & 0
```

x07)

- **opcode = 0b0011**: isto é uma configuração de rede
- **opcode<<4**: Empurra os 4 bits do opcode para a esquerda do byte, ficamos com  
0011 xxxx.
- **0b0<<3**: Deixa 1 bit a zero na posição 3.
- **target\_net\_id & 0x07**: 0x07 em binário é 0000 0111. Garantimos que a rede de destino cabe me 3 bits.
- **Comando (|)**: Ficamos com [4 bits Opcode] [1 bit zero] [3 bits TargetNetID].

```
mac_bytes = struct.pack('>I', mac_address_int)
```

O MAC Address é um número grande (32 bits). Em vez de estar a fazer a matemática "à mão", ao usar o `struct.pack('>I', ...)`, convertemos logo esse número grande em 4 bytes separados e já ordenados em Big-Endian.

```
byte6 = ((data_ch & 0x07) << 5) | (0b0 << 4) | ((ctrl_ch & 0x07) << 1) | 0b0
```

- **(data\_ch & 0x07) << 5**: Protege o canal de dados (máximo 7) e empurra-o 5 posições para a esquerda.
- **0b0 << 4**: Põe um bit a 0 no meio.
- **(ctrl\_ch & 0x07) << 1**: Protege o canal de controlo e empurra-o 1 posição para a esquerda.
- **| 0b0**: Cola um bit a zero no fim (na posição 0).
- Ficamos com [3 bits Data Ch] [1 bit 0] [3 bits Ctrl Ch] [1 bit 0].

